# Issue #307 Development Plan
# Remove the Remaining Ambient Variable-Level Interface from Tensor-Native Operator and Transfer Realization

June 2026

## 1 Objective

The purpose of issue #307 was to remove the remaining dependence of the tensor-native operator, gather/scatter, and transfer layers on an *ambient variable-level flat tensor block* as the primary coefficient carrier.

After issue #305, the exact local factor-space semantics were already corrected:

- `TensorDofLayout` describes indexing conventions rather than exact local spaces,
- exact local shapes come from volume and skeleton contexts,
- and local matrix-free realization uses those exact shapes honestly.

What remained at the start of #307 was the global/operator/transfer-facing interface. In particular, that interface still relied substantially on:

- `ambientShape(...)` queries,
- flat offset-based gather/scatter,
- fixed operator input/output sizes derived from one ambient variable block,
- and transfer composition built from the same ambient block semantics.

The implemented work under #307 replaced that remaining ambient interface as the *primary operator-side coefficient carrier* with an explicit tensor-native coefficient-indexing story. The final implementation now provides:

- `TensorCoefficientMap` as the explicit coefficient-indexing layer,
- explicit coefficient-container counts for realized operators and transfer metadata,
- trace coefficient sharing semantics keyed to underlying trace support,
- a component-factorized and lazy indexing implementation rather than an eager tensor-entity/global-key map,
- `TensorCoefficientTraversal` as a compact per-entity traversal descriptor,
- traversal-aware gather/scatter support,

- and traversal/scalar coefficient access in the realized matrix-free tensor operator.

At the same time, the present implementation preserves a compatibility boundary in transfer apply: transfer row/column metadata now uses the explicit coefficient-container interface, but the current placeholder tensor-product transfer apply path still operates internally on the ambient tensor-product compatibility container.

## 2 Design interpretation after clarification

The most important clarification for this issue is that the redesign is *not* merely local or single-process in spirit. Even if the initial test coverage remains single-rank, the intended abstraction should already be shaped for MPI-distributed tensor-native discretization.

The target ownership model is:

- each MPI process owns a contiguous portion of each component mesh,
- and therefore owns a corresponding structured portion of the tensor mesh.

This does *not* require that the present implementation solve a full distributed parallel numbering problem in one step, but it does require that the new abstractions be compatible with such a direction rather than secretly encoding a process-local-only worldview.

A second crucial clarification concerns traces. The current tensor skeleton is slice-based, and that is acceptable as a *realization carrier*. However, for coefficient indexing, the intended mathematical semantics are different:

- multiple tensor skeleton entities may "see" the same trace support location,
- but there should be only one global coefficient associated with that trace degree of freedom.

Thus, the redesign should distinguish carefully between:

- tensor skeleton entities used for lower-dimensional integration and boundary realization, and
- global coefficient entities used for trace coefficient ownership and movement.

This is likely the first place where the current slice-based structural model will need a coefficient-side correction.

## 3 Guiding design principle

The intended principle for this issue is the following:

> Exact local realization should continue to be driven by tensor cell tuples and tensor skeleton entities, while global coefficient movement should be driven by explicit ownership-aware coefficient index sets rather than by one ambient per-variable tensor block.

Equivalently:

- local realization contexts determine *which local basis tuples are admissible*,
- but coefficient movement should determine *which global coefficients participate in that entity*,
- and these should be related by explicit index maps rather than by offsetting into a single variable-level ambient flat vector.

## 4 Scope and intended end state

By the end of this issue, we want the following to be true:

1. Realized operators no longer depend primarily on `TensorDofLayout::ambientShape(...)` for their coefficient interface.
2. Gather/scatter is no longer fundamentally flat-offset-based for operator use; instead, it works through explicit index lists associated with the current realized entity.
3. Transfer uses the same redesigned coefficient-indexing story rather than tensor-product composition over one ambient variable block.
4. `TensorOperator::domainSize()` and `rangeSize()` remain available, but are understood as sizes of the new coefficient-indexed container interface rather than as sizes of an ambient variable-local tensor envelope.
5. The resulting design is shaped so that MPI-aware ownership and indexing can be expressed naturally, even if current tests remain single-rank.

We do *not* require complete removal of every ambient-shape helper from the codebase. Such helpers may remain as secondary compatibility utilities where they are still useful internally. The key requirement is that they cease to define the primary operator/transfer/gather-scatter semantics.

## 5 Primary architectural correction

This issue appears to require a new explicit coefficient-indexing layer.

The likely role of that layer is to answer questions such as:

- Which global coefficient indices belong to this trial variable on this tensor cell tuple?
- Which global coefficient indices belong to this trace variable on this tensor skeleton realization entity?
- Which of those trace coefficients are shared across multiple slice-based skeleton entities because they correspond to the same underlying trace support location?
- What is the total owned/global coefficient count for a variable in a given role?

The exact naming may vary, but conceptually we need something like a tensor-native global coefficient map.

## 6 Trace-specific design correction

A particularly important part of this issue is to avoid confusing:

- *skeleton realization entities*, and
- *trace coefficient entities*.

The current slice-based tensor skeleton semantics are still useful:

- they tell us where lower-dimensional boundary integration occurs,
- they distinguish retained-component cell tuples incident on a chosen orthogonal component vertex,
- and they support the existing trace-aware local apply path.

However, coefficient ownership should not duplicate trace coefficients merely because several such realization entities touch the same underlying trace support location.

The intended correction is therefore:

- keep slice-based tensor skeleton entities as the geometric carriers for boundary realization,
- but define trace coefficient indexing in terms of a more merged support notion,
- likely keyed by the distinguished trace-component support location together with retained-component basis structure,
- so that multiple realization entities may map to the same global trace coefficient indices where mathematically appropriate.

We should expect the first implementation of this to be modest and explicit rather than fully abstracted in the most general possible form.

# 7 Implemented strategy and resulting architecture

The implementation proceeded in two distinct internal stages: an initial explicit coefficient-indexing pass, followed by a corrective redesign that removed the remaining eager tensor-entity expansion from the new indexing layer.

## 7.1 Step 1: Explicit coefficient indexing support

The first major step was the introduction of `TensorCoefficientMap` as the explicit tensor-native coefficient-indexing abstraction.

The earliest version of that abstraction provided explicit per-entity global index lists, which was already a substantial improvement over ambient per-variable tensor blocks. However, that first design still expanded tensor cells and tensor skeleton entities too eagerly. During the same issue, that design was corrected.

The resulting implementation is now intentionally *component-factorized and lazy.* In particular:

- for volume variables, global coefficient indices are derived from per-component active-cell / local-basis coordinates,
- for trace variables, the distinguished trace component contributes a support-vertex coordinate while retained components contribute per-cell / local-basis coordinates,
- coefficient counts are derived from component-factor sizes rather than from one ambient variable-level tensor block,
- and trace coefficient sharing is enforced naturally through trace-support coordinates, so that multiple slice-based tensor skeleton entities may map to the same underlying trace coefficients.

Thus, the implemented coefficient map no longer stores eager tensor-cell-expanded or tensor-skeleton-expanded global maps as its primary representation.

### 7.2 Step 2: Gather/scatter redesign

`TensorGatherScatter` was redesigned so that the operator-facing tensor-native path no longer depends primarily on flat offset-based ambient-block movement.

The implemented work provides:

- explicit global-index-list gather/scatter,
- compatibility shape+offset helpers retained only as shims,
- entity-aware gather/scatter for tensor cell tuples and tensor skeleton entities,
- and traversal-aware gather/scatter through `TensorCoefficientTraversal`.

This means that coefficient movement can now be driven either by explicit indices or by a traversal description of one realized tensor entity, rather than by one flat per-variable ambient block.

### 7.3 Step 3: Traversal support and operator migration

A further refinement beyond the initial explicit index-list story was the introduction of `TensorCoefficientTravers`

This object provides a compact descriptor of the coefficient-access pattern for one realized tensor entity. It records:

- the exact local factor-space shape,
- component-factor base coordinates,
- component strides in the global coefficient container,
- and, for trace traversals, the distinguished trace-support substitution information.

`TensorCoefficientMap` now produces:

- volume traversals for tensor cell tuples,
- and skeleton traversals for tensor skeleton entities.

The realized matrix-free tensor operator was then migrated beyond explicit per-entity index vectors. In particular, it now uses traversal/scalar coefficient access directly in:

- the volume apply path,
- and the trace-aware boundary apply path.

The local finite-element-aware integration logic from #299/#302/#304/#305 remains the same in substance. What changed was the coefficient-access interface surrounding it.

The old vector-returning helper methods remain available, but they are now compatibility wrappers implemented through traversal/scalar indexing rather than through eager global maps.

### 7.4 Step 4: Transfer redesign boundary reached

`TensorTransferOperator` was partially migrated to the new coefficient-container semantics.

In particular, the implementation now reports:

- transfer row counts, and
- transfer column counts

through explicit coefficient-container counts derived from `TensorCoefficientMap`.

However, the present placeholder tensor-product transfer `apply()` path still operates internally on the ambient tensor-product compatibility container. This was left intentionally as a compatibility boundary because:

- the current component-transfer realization remains mathematically modest,
- the current transfer apply path is still a placeholder tensor-product composition,
- and a fully traversal-/block-aware transfer apply path is better treated as a follow-on rather than silently folded into this issue.

Thus, the metadata/interface part of the transfer redesign was completed here, while the internal traversal-/block-aware transfer realization remains future work.

### 7.5 Step 5: Tests and semantic updates

The tests were updated so that they no longer encode ambient tensor-block sizing as the intended tensor-native semantics.

The final test coverage added under this issue now includes checks for:

- explicit coefficient-container counts,
- lazy scalar local-ordinal-to-global-index behavior,
- traversal/scalar indexing consistency with vector helper semantics,
- traversal-aware gather/scatter for both volume and skeleton entities,
- operator sizing and apply behavior against the explicit coefficient-container interface,
- transfer metadata sizing against the same explicit coefficient-container interface,
- and trace coefficient sharing semantics for slice-based skeleton entities touching the same trace support.

## 8 Files and subsystems actually involved

The implemented work touched the following main files and subsystems:

- `src/include/TensorDofLayout.h`
- `src/Mesh/TensorDofLayout.cpp`
- `src/include/TensorCoefficientMap.h`

- `src/Mesh/TensorCoefficientMap.cpp`
- `src/include/TensorCoefficientTraversal.h`
- `src/Mesh/TensorCoefficientTraversal.cpp`
- `src/include/TensorGatherScatter.h`
- `src/Mesh/TensorGatherScatter.cpp`
- `src/include/TensorOperator.h`
- `src/Problem/TensorOperator.cpp`
- `src/include/TensorTransferOperator.h`
- `src/Mesh/TensorTransferOperator.cpp`
- `src/include/ComponentTransferOperator1D.h`
- `src/Mesh/ComponentTransferOperator1D.cpp`
- `src/include/ComponentMeshTopology1D.h`
- `src/Mesh/ComponentMeshTopology1D.cpp`
- unit tests for layout, gather/scatter, operator, transfer, tensor mesh, and tensor topology.

The main new header/implementation pairs added during this work were:

- `TensorCoefficientMap.h/.cpp`
- `TensorCoefficientTraversal.h/.cpp`

## 9 Validation strategy actually used

The validation strategy remained incremental and test-driven throughout the implementation.

In practice, the work proceeded roughly in this order:

1. add tests for explicit coefficient-container counts and lazy scalar local-ordinal-to-global-index behavior,
2. add tests for trace coefficient sharing semantics,
3. migrate gather/scatter and operator sizing to the explicit coefficient-container interface,
4. update the explicit-reference operator tests so that they compare against the new coefficient-indexing semantics rather than ambient tensor-block semantics,
5. refine the coefficient map from an eager explicit-entity expansion to a component-factorized and lazy implementation,
6. add traversal-specific regression tests,
7. migrate the realized operator from explicit per-entity index vectors to traversal/scalar coefficient access,

8. align transfer metadata sizing with the same explicit coefficient-container story,

9. and then run the broader tensor-native test subset, including the MPI test run on `-group=Tensor*`.

The operator correctness tests in `TensorOperatorTests.cpp` remained a key validation anchor. Those tests now compare realized operator application against an explicit reference that follows the new coefficient-indexing semantics. Additional focused tests in `TensorGatherScatterTests.cpp` now pin traversal/scalar indexing behavior and traversal-aware gather/scatter consistency directly.

## 10 Remaining risks and pressure points

The main remaining risks after the implemented work are as follows.

- **The current traversal layer is still scalar/traversal-oriented rather than yet fully block-/range-aware.** This is the right next structural step, but it has not yet been pushed into explicit contiguous-range or strided-block packaging for MPI-aware communication.
- **Transfer and operator remain at slightly different maturity levels.** Operator application now uses traversal/scalar coefficient access directly, whereas transfer currently uses the explicit coefficient-container story mainly at the metadata level and still retains an ambient compatibility path internally in `apply()`.
- **MPI-aware ownership has been shaped but not implemented.** The current component-factorized and traversal-oriented design is meant to support contiguous componentwise ownership later, but the present tests still operate on flat coefficient containers without a full distributed coefficient-management layer.
- **Component-factorized indexing still depends on the current component-topology capabilities.** This is now dramatically better than eager tensor-entity expansion, but additional follow-on work may still be useful if we later want stronger block/range decomposition or ownership-aware traversal partitioning.

## 11 Conditions that would force a stop-and-ask event

Under the present workflow, we should stop and ask only if:

- additional required files or implementation context turn out to be missing, or
- the trace coefficient semantics appear to require a substantially larger architectural correction than the present issue description supports.

At present, we do *not* appear to be blocked by missing files, and the design direction seems coherent enough to begin implementation.

## 12 Implemented outcome

The implemented outcome of #307 is now:

- a tensor-native coefficient-indexing layer that replaces the ambient variable-level block as the primary operator-side coefficient carrier,

- a component-factorized and lazy `TensorCoefficientMap` implementation,
- traversal-aware gather/scatter in addition to explicit index-list movement,
- realized operator application that now uses traversal/scalar coefficient access directly while retaining the exact local-context semantics established in #305,
- transfer row/column metadata aligned with the same explicit coefficient-container story,
- and updated tests/documentation reflecting the corrected separation among exact local factor-space realization, global coefficient indexing/movement, and per-entity traversal structure.

The main architectural result is that the tensor-native stack now distinguishes four roles more clearly than before:

- geometric realization entities,
- exact local factor spaces,
- global coefficient indexing and movement,
- and the traversal structure of one realized tensor entity through that coefficient space.

The most natural follow-on beyond the present implementation is to use that traversal structure more directly for range-/block-aware gather/scatter and eventual MPI-aware ownership/communication based on contiguous componentwise index blocks.